

Secure Kubernetes Deployment

A Production-Aligned DevSecOps Workflow

Technical Report, Reproduction Runbook and Evidence Record

Author	Olalekan Ismail Musa
Repository	github.com/Lakewest1/Secure-Kubernetes-Deployment
Version / Date	2.0 · October 2026
Classification	Production-aligned security engineering demonstration (not a production system)

A security-first container and Kubernetes workflow on Microsoft Azure. Source code is scanned, built into a container image, authenticated to Azure without stored secrets, published to a registry, deployed to a hardened Kubernetes cluster, policy-checked at admission, validated and observed. Pipeline publishing is automated; deployment to the cluster is manual.

Trivy

Gitleaks

GitHub Actions

GitHub OIDC

Azure Container Registry

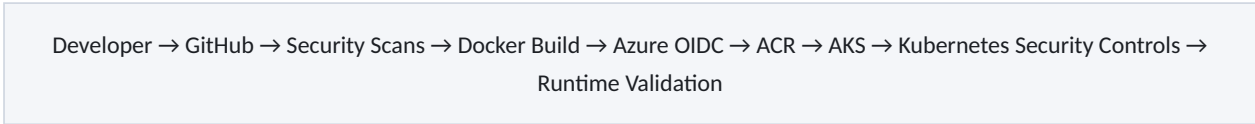
AKS

Cilium

OPA Gatekeeper

Kubernetes RBAC

NGINX



Contents

How to Read This Report	4
1. Executive Summary.....	5
2. Project Scope and Objective	6
3. Final Architecture	7
4. Technology Stack.....	7
5. Azure Environment	8
6. Project Evolution (Historical).....	8
7. Containerization and Vulnerability Remediation	9
8. Final Container Hardening	10
9. Image Identity: SHA Tags and Digests	11
10. GitHub Actions DevSecOps Pipeline.....	12
11. GitHub OIDC Federation.....	12
12. ACR Identity Model (AcrPush vs AcrPull)	13
13. Kubernetes RBAC	13
14. NetworkPolicy	14
15. Cilium and AKS Networking	14
16. OPA Gatekeeper	15
17. Gatekeeper Negative Test	15
18. Gatekeeper Audit Findings.....	16
19. Final Kubernetes State, Service and Secret.....	16
20. Evidence Strategy and Screenshot Plan	17
21. ACR Observability Investigation	19
22. Initial vs Final State	20
23. Challenges and Troubleshooting	20
24. Security Validation Matrix.....	21
25. Reproduction Runbook	23
26. Command Reference.....	25
27. Configuration Reference	27
28. Production Readiness Gap Analysis	28
29. Future Roadmap	29
30. Interview Defense Guide.....	30

31. Portfolio Value..... 31

32. Conclusion..... 32

33. Reality Audit..... 32

34. Evidence Index 37

How to Read This Report

Every statement is tagged by status. The categories are never mixed.

Tag	Meaning
IMPLEMENTED	Built and present in the final environment
TESTED	Implemented and exercised with a specific command or test
OBSERVED	Seen in output or telemetry; interpretation is stated separately
HISTORICAL	Part of earlier project phases; not the final state
PLANNED	Not implemented. Listed only under Future Improvements

IMPORTANT

Where this report gives a command that was not part of the recorded project, it is marked **Standard, verify**. Verify it against your repository files and current vendor documentation before relying on it. Where an actual result is not recorded, the report says "Not recorded" rather than inventing one.

EVIDENCE

No screenshots were attached to the material used to write this report. Evidence IDs (EVID-xx) are therefore **slots to be filled with your real screenshots**. Each slot states what the screenshot is expected to show. Before inserting an image, confirm it actually displays that content (see Section 20).

Technologies in Plain English

Technology	What it is	Role here
Trivy	Open-source scanner that finds known vulnerabilities and misconfigurations in files and container images.	Scans the repository and the built image in CI.
Gitleaks	Tool that searches code and history for committed secrets such as keys and tokens.	Runs in the first pipeline job.
GitHub OIDC	A way for GitHub Actions to prove its identity to a cloud provider with a short-lived token instead of a stored password.	Authenticates the workflow to Azure.
Azure Container Registry (ACR)	Azure service that stores container images.	Holds <code>secure-nginx</code> images tagged by commit SHA.
Azure Kubernetes Service (AKS)	Managed Kubernetes cluster on Azure.	Runs the hardened NGINX workload.
Kubernetes RBAC	Rules controlling which identities can perform which actions on which Kubernetes resources.	<code>nginx-sa</code> may only get/list pods.
NetworkPolicy	Kubernetes rules that filter network traffic to and from pods.	Allows TCP 8080 to the NGINX pods; limitations are stated.
Cilium	Networking component that enforces NetworkPolicy and handles packet forwarding using eBPF.	Enabled as AKS policy engine and dataplane.

OPA Gatekeeper	Kubernetes admission-control framework that enforces policies before resources are accepted.	Rejects Deployments without <code>runAsNonRoot: true</code> .
Rego	The policy language used by Open Policy Agent to express rules.	Defines the non-root rule in the ConstraintTemplate.

Key Controls at a Glance

Control	What	Why	Implementation	Validation	Limitation
GitHub OIDC	Passwordless auth from GitHub Actions to Azure	Avoid a long-lived Azure client secret	Federated credential on managed identity, pinned to repo and <code>main</code> ; role <code>AcrPush</code>	Pipeline authenticated and pushed to ACR	Depends on protection of <code>main</code>
SHA-tagged image	Image tagged with commit SHA, digest recorded	Reproducibility, rollback, audit	CI tags with commit SHA	Tag and digest recorded for deployed image	ACR tags are mutable unless locked
Container hardening	Non-root, no privilege escalation, read-only rootfs	Limit damage if NGINX is compromised	<code>securityContext</code> plus <code>emptyDir</code> for <code>/tmp</code> and <code>/var/cache/nginx</code>	2/2 pods Running, 0 restarts	Single node; no Pod Security Standards
Kubernetes RBAC	Dedicated ServiceAccount with a narrow Role	Least-privilege workload identity	<code>nginx-sa</code> , <code>nginx-role</code> , <code>nginx-binding</code>	<code>can-i</code> : yes, yes, no, no	App does not need API access
NetworkPolicy	Allow TCP 8080 to NGINX pods	Demonstrate policy enforcement	<code>nginx-allow-http</code> on Cilium	Policy present and described	Ingress from all sources; egress unrestricted
OPA Gatekeeper	Admission policy requiring non-root	Block non-compliant workloads	Template <code>k8srequirednonroot</code> , constraint <code>require-non-root</code>	Test Deployment denied; NotFound	Broad scope; 5 audit findings on system workloads
ACR telemetry	Metric-level review of registry activity	Understand publish activity and 404s	Reviewed 24-hour metrics	91 transactions, 4 pushes, 15 HeadBlob 404	No caller identity or IP; no attribution

1. Executive Summary

This project began as a local Docker and Minikube lab: build an NGINX container, scan it with Trivy, reduce the findings, push to Docker Hub and run it with basic Kubernetes hardening. It has since been rebuilt as an Azure workflow:

```
Build -> Scan -> Remediate -> Authenticate -> Publish -> Deploy -> Harden -> Enforce -> Validate -> Observe
```

What is IMPLEMENTED and TESTED in the final state:

- A GitHub Actions pipeline with three visible jobs: filesystem and secret scans (Trivy, Gitleaks), build and container image scan (Docker, Trivy), and push to Azure Container Registry.

- Passwordless CI authentication to Azure using GitHub OIDC and a user-assigned managed identity holding only `AcrPush` on the registry. No long-lived Azure client secret is stored.
- Commit-SHA image tags, with the deployed image also identified by digest.
- An AKS cluster using Azure CNI overlay with the Cilium network policy engine and dataplane.
- A hardened NGINX Deployment (non-root, no privilege escalation, read-only root filesystem, resource limits, probes, 2 replicas).
- Least-privilege Kubernetes RBAC, validated with `kubectl auth can-i`.
- An OPA Gatekeeper admission policy requiring `runAsNonRoot: true`, with a **negative test** in which a non-compliant Deployment was rejected.
- An investigation of ACR metric-level telemetry, interpreted conservatively.

What is NOT implemented (stated plainly):

- The pipeline **does not deploy to AKS**. It publishes the validated image to ACR. Deployment to AKS is performed manually with `kubectl`.
- The NetworkPolicy allows ingress from all sources to TCP 8080 and leaves egress unrestricted. It is not a default-deny or zero-trust design.
- The Kubernetes Secret is demonstration data (base64 `Hello`), not a secret management design.
- No image signing, SBOM, runtime monitoring, Prometheus/Grafana, Kyverno, Infrastructure as Code, or environment separation.

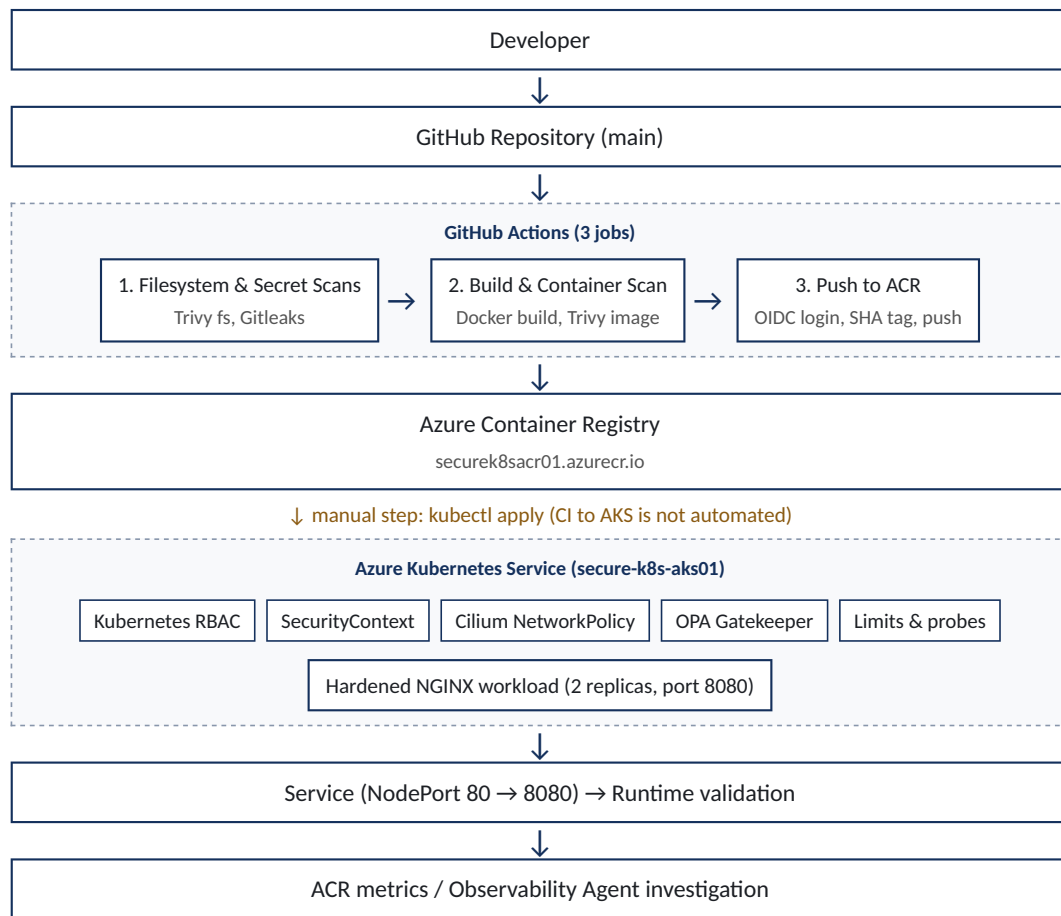
2. Project Scope and Objective

Objective: Demonstrate a complete, security-first container and Kubernetes workflow covering build, scan, remediate, authenticate, publish, deploy, harden, enforce, validate and observe.

Scope: A static HTML application served by NGINX. The application is deliberately simple so that attention stays on the delivery pipeline and the platform controls around it.

Out of scope: Application-layer logic, TLS ingress, multi-environment promotion, production secret management, runtime threat detection.

3. Final Architecture

**IMPORTANT**

The break between ACR and AKS is intentional documentation of reality. Image publishing is automated; cluster deployment is manual.

4. Technology Stack

Layer	Technology	Status
Application	HTML, NGINX	IMPLEMENTED
Container	Docker	IMPLEMENTED
Security scanning	Trivy (filesystem and image), Gitleaks	IMPLEMENTED, TESTED
CI/CD	GitHub Actions	IMPLEMENTED
Cloud identity	GitHub OIDC, Azure user-assigned managed identity	IMPLEMENTED
Cloud	Microsoft Azure	IMPLEMENTED
Registry	Azure Container Registry	IMPLEMENTED
Kubernetes	AKS, Kubernetes RBAC, SecurityContext, NetworkPolicy, EndpointSlice, Secrets	IMPLEMENTED
Networking	Cilium network policy, Azure CNI overlay	IMPLEMENTED
Policy-as-code	OPA Gatekeeper, Rego	IMPLEMENTED, TESTED

Layer	Technology	Status
Observability	ACR metrics, Azure Observability Agent	OBSERVED

No other technologies are claimed. Docker Hub, Minikube and Snyk are **HISTORICAL** only.

5. Azure Environment

Item	Value
Subscription	SOC-lab
Resource group	SOC-lab-01
Region	East US
AKS cluster	secure-k8s-aks01
ACR	securek8sacr01
ACR login server	securek8sacr01.azurecr.io
Kubernetes version (during validation)	1.35.8
Node pool	1 x Standard_D2_v3
AKS features	Azure RBAC enabled; Microsoft Entra managed identity enabled; OIDC issuer enabled

Final AKS networking

Setting	Value	Meaning
NetworkPlugin	azure	Pod networking is provided by Azure CNI.
NetworkPluginMode	overlay	Pods receive IPs from a private overlay CIDR rather than from the VNet subnet, which conserves VNet address space. Node-to-pod traffic is encapsulated.
NetworkPolicy	cilium	Kubernetes NetworkPolicy objects are enforced by Cilium.
NetworkDataplane	cilium	Cilium (eBPF) handles packet forwarding and policy in place of the default dataplane.

EVIDENCE

EVID-07 should show the AKS network profile (plugin, plugin mode, policy, dataplane).

6. Project Evolution (Historical)

Stage	What happened	Status
Local lab	Docker build, Trivy scan, Debian-based NGINX image	HISTORICAL
Registry	Image pushed to Docker Hub	HISTORICAL
Runtime	Minikube with <code>kubectl port-forward</code>	HISTORICAL
Cloud scan	Snyk scan of the image (reported 90 issues: 1 high, 5 medium, 84 low)	HISTORICAL. Not continuous monitoring, and not the final state.

Stage	What happened	Status
Azure move	AKS + ACR, GitHub Actions, OIDC	CURRENT
Hardening	RBAC, NetworkPolicy, Cilium, Gatekeeper	CURRENT

The original report claimed Snyk continuous monitoring, a "zero trust" network policy, and a Prometheus/Grafana, EKS, Kyverno and Sentinel roadmap. None of these are part of the final implementation. Roadmap items appear only in Section 29.

7. Containerization and Vulnerability Remediation

7.1 Initial image (HISTORICAL)

```
docker build -t lakewest/my-nginx-appl:latest .
trivy image lakewest/my-nginx-appl:latest
```

The initial image was Debian-based and Trivy reported many OS-level vulnerabilities. Scanning was slow (about one hour in the local lab), attributed to the vulnerability database download and scanning overhead.

7.2 Findings and root cause

Root-cause analysis found unnecessary packages and transitive dependencies in the image, for example `libtiff`, `libheif` and `libxslt`, none of which a static NGINX site needs. Examples of findings from that original analysis:

CVE	Component (as originally analyzed)
CVE-2026-4105	systemd
CVE-2026-1965	curl
CVE-2026-0990	libxml2
CVE-2026-27171	zlib

WARNING

These CVEs belonged to the **initial image** and were part of the remediation investigation. They must not be read as present in the final image.

7.3 Remediation path

```
Initial image -> Trivy findings -> Root-cause analysis -> Smaller Alpine-based NGINX image
-> Removal of unnecessary dependencies -> Reduced attack surface -> Follow-up scanning
```

The project observed roughly an **80% reduction** in reported vulnerabilities after image optimization.

IMPORTANT

This percentage comes from the project's observed scan comparison. Before treating it as a current benchmark, reproduce it with the same scanner version, vulnerability database snapshot and image versions. Vulnerability counts change as databases update.

SECURITY NOTE

A smaller image is not automatically secure. Alpine does not make an image safe by itself. The improvement came from **removing unneeded packages and dependencies and then re-scanning** to validate the result. Alpine uses musl libc and its own package set, which can have different compatibility characteristics and its own vulnerabilities.

EVIDENCE

EVID-01 (initial scan) and EVID-02 (remediated scan) support this section. Compare them only if both show scanner version and image name legibly.

8. Final Container Hardening

8.1 securityContext (IMPLEMENTED, enforced by Gatekeeper for `runAsNonRoot`)

```
securityContext:
  runAsNonRoot: true
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
```

Control	What it does	Why it matters here
<code>runAsNonRoot: true</code>	Kubelet refuses to start a container that would run as UID 0.	A compromise of NGINX does not yield root inside the container.
<code>allowPrivilegeEscalation: false</code>	Sets <code>no_new_privs</code> , blocking setuid binaries and similar escalation.	Reduces in-container privilege escalation paths.
<code>readOnlyRootFilesystem: true</code>	Mounts the container root filesystem read-only.	Prevents an attacker from modifying binaries or dropping files in the image layers.

8.2 Writable volumes

Two `emptyDir` volumes are mounted:

- `/tmp`
- `/var/cache/nginx`

NGINX writes temporary and cache data (for example proxy/client body temp paths and PID or cache files) at runtime. With a read-only root filesystem these locations would fail. Mounting small, ephemeral, writable volumes only where needed keeps the rest of the filesystem immutable.

VERIFY

Which exact paths your NGINX configuration writes to depends on the image and `nginx.conf`. The two paths above are the ones used in this project.

8.3 Replicas, resources and probes

Setting	Value
Replicas	2

Setting	Value
CPU request / limit	250m / 500m
Memory request / limit	64Mi / 128Mi
Liveness probe	HTTP GET / on port 8080
Readiness probe	HTTP GET / on port 8080

Why these are security and reliability controls:

- **Limits** cap CPU and memory so a runaway or abused container cannot starve neighbours on the node (a denial-of-service containment measure).
- **Requests** let the scheduler place pods predictably.
- **Liveness** restarts a hung container; **readiness** keeps a pod out of Service endpoints until it can serve, so traffic is never sent to a broken pod.
- **Two replicas** give basic availability during a pod restart. On a single node this is not node-level high availability.

PRODUCTION GAP

The cluster has one node. Replica count does not protect against node failure.

9. Image Identity: SHA Tags and Digests

Final deployed image:

```
securek8sacr01.azurecr.io/secure-nginx:f12214058cd0e11d8b77dbe5ce0de799a777801f
```

Digest:

```
sha256:f95311cd1cc733c91c5b67b26c15f5dde5369cc1d6f8075871cfd9a96df93880
```

Concept	Explanation
Why not <code>latest</code>	<code>latest</code> is a mutable pointer. The same reference can resolve to different content over time, which caused the stale-HTML problem in the early lab (Section 23).
Why a commit-SHA tag	The tag maps one-to-one to a Git commit, so any deployed image traces back to exact source.
Why a digest	A digest is a content hash of the manifest. It cannot be repointed, so it is a stronger immutability guarantee than any tag.
Rollback	Redeploying a previous SHA tag or digest restores a known artifact deterministically.
Auditability	Incident review can answer "what exactly ran" by commit and digest.

SECURITY NOTE

Tags in ACR are mutable unless tag locking or similar is configured. The SHA convention reduces risk; the digest is what guarantees identity. Referencing the digest in the Deployment (`image: ...@sha256:...`) is the strongest form. **Verify** which form your `K8s/deployment.yaml` uses and record it here.

10. GitHub Actions DevSecOps Pipeline

Filesystem & Secret Scans -> Build & Container Scan -> Push to Azure Container Registry

Job	Steps	Control
1. Filesystem & Secret Scans	Trivy filesystem scan, Gitleaks	Catch vulnerable dependencies/config and committed secrets before building
2. Build & Container Scan	Docker build, Trivy image scan	Scan the actual built artifact
3. Push to Azure Container Registry	Azure login (OIDC), ACR login, SHA tag, ACR push	Publish only after earlier jobs pass

Workflow permissions:

```
permissions:
  contents: read
  id-token: write
```

- `contents: read` limits the `GITHUB_TOKEN` to reading the repository.
- `id-token: write` lets the job **request** a signed OIDC token from GitHub. Without it, `azure/login` cannot obtain the token to exchange with Azure. It does not grant write access to the repository.

No long-lived Azure secret. The workflow does not use an Azure client secret. GitHub repository values used are identifiers (client ID, tenant ID, subscription ID), not credentials.

VALIDATION

Pipeline success is shown in EVID-05. Job-level failure behavior (for example, whether a scan failure blocks the push) depends on the workflow's `needs: and Trivy exit-code` settings. **Verify** and record them from the workflow file; this report does not claim a specific severity threshold.

11. GitHub OIDC Federation

Item	Value
Managed identity	<code>github-actions-secure-k8s</code>
Federated credential	<code>github-actions-main</code>
Issuer	<code>https://token.actions.githubusercontent.com</code>
Subject	<code>repo:Lakewest1/Secure-Kubernetes-Deployment:ref:refs/heads/main</code>
Audience	<code>api://AzureADTokenExchange</code>
ACR role	<code>AcrPush</code>

Flow:

1. The job requests a short-lived OIDC token from GitHub (enabled by `id-token: write`).
2. The token carries claims, including the subject (repo and branch).
3. Azure validates the token against the federated credential: issuer, subject and audience must match exactly.

4. Azure issues access for the managed identity, limited to its role assignments.
5. The workflow pushes to ACR.

Why this is preferable to a stored service principal secret: nothing reusable is stored in GitHub; tokens are short-lived; there is no secret to rotate or leak; and the **subject restricts use to the `main` branch of this repository**. A pull request branch or a different repository does not match the subject.

SECURITY NOTE

Because the subject is pinned to `refs/heads/main`, only workflows running on `main` can obtain the identity. Protect `main` with branch protection to make that boundary meaningful (Section 28).

12. ACR Identity Model (AcrPush vs AcrPull)

Identity	Role	Purpose
GitHub Actions managed identity (<code>github-actions-secure-k8s</code>)	<code>AcrPush</code>	Push images to ACR
AKS kubelet identity	<code>AcrPull</code>	Pull images to run workloads

Why they are not merged: the CI identity needs to write; the cluster needs only to read. Combining them would give the cluster node identity push rights (so a compromised node could tamper with images) or give CI broader rights than it needs. Separate identities with single-purpose roles are a least-privilege decision. A pull-only runtime identity also limits the blast radius of node compromise.

NOTE

`AcrPush` in Azure includes pull capability. The point of the separation is that the **runtime** identity has no push rights.

13. Kubernetes RBAC

Object	Name	Detail
ServiceAccount	<code>nginx-sa</code>	Namespace <code>default</code>
Role	<code>nginx-role</code>	<code> pods: get, list</code>
RoleBinding	<code>nginx-binding</code>	Binds <code>nginx-sa</code> to <code>nginx-role</code>

Validation (TESTED)

Command	Result
<code>kubectl auth can-i get pods --as=system:serviceaccount:default:nginx-sa</code>	yes
<code>kubectl auth can-i list pods --as=system:serviceaccount:default:nginx-sa</code>	yes
<code>kubectl auth can-i delete pods --as=system:serviceaccount:default:nginx-sa</code>	no
<code>kubectl auth can-i create deployments --as=system:serviceaccount:default:nginx-sa</code>	no

Least privilege: the account holds only the verbs and resources listed, scoped to one namespace through a Role (not a ClusterRole). Write verbs and unrelated resources are denied.

IMPORTANT

The static NGINX application does **not** require Kubernetes API access. The ServiceAccount and Role demonstrate restricted workload identity and least-privilege Kubernetes authorization.

EVIDENCE

EVID-08 should show the four `auth can-i` outputs.

14. NetworkPolicy

Field	Value
Name	<code>nginx-allow-http</code>
Pod selector	<code>app=nginx-deploy</code>
Allowed port	TCP 8080
Ingress sources	All sources
Egress	Unrestricted

WARNING

This is **not** zero trust and **not** "deny all by default". The rule allows ingress from any source to TCP 8080 on the selected pods, and egress is not restricted. What it demonstrates is that NetworkPolicy objects are accepted and enforced by the Cilium-backed AKS cluster. Selecting pods with an ingress policy does make unlisted ingress ports non-allowed for those pods, but this is port filtering, not source-based segmentation.

PRODUCTION GAP

A production implementation should restrict ingress to known sources (for example an ingress controller namespace), add an explicit default-deny posture per namespace, and control egress (DNS and required destinations only).

15. Cilium and AKS Networking

Cilium was enabled through AKS networking configuration, not by installing a separate CNI. The migration command used:

```
az aks update `
  --resource-group "SOC-lab-01" `
  --name "secure-k8s-aks01" `
  --network-policy cilium `
  --network-dataplane cilium
```

Observed side effect: the Service disappeared during the migration and had to be reapplied:

```
kubectl apply -f K8s/service.yaml
```

Validation:

```
kubectl get networkpolicy
kubectl describe networkpolicy nginx-allow-http
kubectl get endpointslice -l kubernetes.io/service-name=nginx-service
```

EndpointSlice vs Endpoints: the core `Endpoints` API is deprecated in favour of `EndpointSlice`, which splits endpoints into multiple objects, scales better, and carries richer data (conditions, topology). Validating against `EndpointSlice` confirms the Service has ready backing pods without using a deprecated API.

WARNING

Plan for Service and workload disruption when changing network policy engine or dataplane on a live cluster. Re-verify Services and policies afterward.

EVIDENCE

EVID-10 (Cilium/policy state) and EVID-14 (Service, EndpointSlice).

16. OPA Gatekeeper

Object	Name
ConstraintTemplate	<code>k8srequirednonroot</code>
Constraint	<code>require-non-root</code>
Requirement	Containers must set <code>securityContext.runAsNonRoot: true</code>

Correction made: the Rego `target` was initially wrong and was changed to:

```
admission.k8s.gatekeeper.sh
```

This is the Gatekeeper admission target. An incorrect target means the template does not evaluate as intended.

Gatekeeper runs as a validating admission webhook: the API server calls it before persisting a resource, and a Rego violation rejects the request.

EVIDENCE

EVID-11 (installation, template and constraint present).

17. Gatekeeper Negative Test

```
kubectl create deployment gatekeeper-test --image=nginx
```

Expected: admission rejected.

Actual (OBSERVED):

```
admission webhook "validation.gatekeeper.sh" denied the request:
[require-non-root] Containers must set securityContext.runAsNonRoot to true
```

Follow-up:

```
kubectl get deployment gatekeeper-test
```

Result: NotFound .

Why this is strong evidence: the policy was not merely installed. It prevented a non-compliant Deployment from being admitted, and the follow-up confirms no object was created.

EVIDENCE

EVID-12 should show both the denial message and the NotFound .

18. Gatekeeper Audit Findings

The constraint matched Deployments broadly and reported **five audit findings** from AKS-managed system workloads:

- konnectivity-agent-autoscaler
- konnectivity-agent
- coredns-autoscaler
- coredns
- cilium-operator

Concept	Meaning
Admission enforcement	Blocks new or updated objects at request time (shown by the negative test)
Audit	Periodically evaluates objects that already exist and reports violations without removing them

These are AKS-managed system workloads and were **not modified**; changing managed components can be reverted by the platform or destabilize the cluster. They are **not application vulnerabilities**. The application Deployment itself satisfied the policy.

PRODUCTION GAP

The constraint scope is broad. A production policy would exclude `kube-system` and other managed namespaces (or scope to application namespaces) so audit output is actionable. Not implemented.

19. Final Kubernetes State, Service and Secret

19.1 Workload state (OBSERVED, final validation snapshot)

Item	Value
Deployment	nginx-deploy , 2/2 Ready
Pods	1/1 Running each, 0 restarts

Validation commands:

```
kubectl get pods -A
kubectl get nodes
kubectl get deployment
kubectl get service
kubectl get endpointslice
```

EVIDENCE

EVID-13 (pods), EVID-14 (service/EndpointSlice).

19.2 Service

Field	Value
Name / type	nginx-service / NodePort
ClusterIP	10.0.230.41
Port / targetPort / NodePort	80 / 8080 / 30007

Path: Client -> Service port 80 -> Pod port 8080.

In the local Minikube phase, access used:

```
kubectl port-forward service/nginx-service 8080:80
```

This was a local access and troubleshooting mechanism, not an exposure design.

PRODUCTION GAP

NodePort is a basic exposure method. Ingress with TLS was not implemented.

19.3 Demonstration Secret

Field	Value
Name	demo-secret
Stored value	SGVsbG8=
Decoded	Hello

SECURITY NOTE

Base64 is encoding, not encryption. Anyone with read access to the Secret can decode it. This is demonstration data and does not represent production secret management.

Future options (PLANNED): Azure Key Vault, Secrets Store CSI Driver, External Secrets, or another approved architecture, plus etcd encryption and RBAC review.

20. Evidence Strategy and Screenshot Plan

Rules: each screenshot gets an ID, a caption, a stated proof claim, a phase, and a reference in the validation matrix. A screenshot is cited only for what is visible in it.

20.1 Folder structure

```
ScreenShot/
|-- 01-Initial-Assessment/
|-- 02-Remediation/
|-- 03-CI-CD/
|-- 04-Azure/
|-- 05-AKS/
|-- 06-Kubernetes-Security/
|-- 07-Gatekeeper/
|-- 08-Validation/
'-- 09-Observability/
```

20.2 Evidence slots

ID	Suggested filename	Caption	Intended to prove (only if visible)
EVID-01	01-Initial-Assessment/EVID-01-initial-trivy-scan.png	Initial Trivy scan of the Debian-based image	Baseline findings before remediation
EVID-02	02-Remediation/EVID-02-remediated-trivy-scan.png	Follow-up Trivy scan of the Alpine-based image	Reduced findings after remediation
EVID-03	02-Remediation/EVID-03-docker-build.png	Docker image build output	Image built successfully
EVID-04	04-Azure/EVID-04-acr-repository-image.png	Image in ACR	Repository, tag and (if shown) digest
EVID-05	03-CI-CD/EVID-05-github-actions-pipeline-success.png	Three-job pipeline completed	Jobs ran and passed
EVID-06	04-Azure/EVID-06-oidc-federated-credential.png	Managed identity federated credential	Issuer, subject, audience settings
EVID-07	05-AKS/EVID-07-aks-network-profile.png	AKS network profile	azure/overlay/cilium/cilium
EVID-08	06-Kubernetes-Security/EVID-08-rbac-can-i.png	kubectl auth can-i results	yes/yes/no/no
EVID-09	06-Kubernetes-Security/EVID-09-networkpolicy.png	describe networkpolicy nginx-allow-http	Selector, port 8080, ingress/egress rules
EVID-10	05-AKS/EVID-10-cilium-migration.png	az aks update for Cilium	Migration executed
EVID-11	07-Gatekeeper/EVID-11-gatekeeper-installed.png	Gatekeeper, template, constraint	Components present
EVID-12	07-Gatekeeper/EVID-12-gatekeeper-denied.png	Denied Deployment and NotFound	Enforcement working
EVID-13	08-Validation/EVID-13-final-pods.png	Final pods and deployment	2/2 Ready, Running, restarts 0
EVID-14	08-Validation/EVID-14-service-endpointslice.png	Service and EndpointSlice	Service present, endpoints ready
EVID-15	09-Observability/EVID-15-acr-observability.png	ACR metrics view	Transactions, pushes, 404 breakdown

If your actual screenshots differ, rename or merge IDs to match what is visible.

21. ACR Observability Investigation

Data available: metric-level telemetry from the Observability Agent. **Not available:** request-level diagnostic logs. Therefore **repository, tag, digest, caller identity and caller IP were not available.**

24-hour observations

Metric	Value
Transactions	91
Pushes (all successful)	4
Pulls (in selected metrics)	0
Data transfer spike	30,374,537 bytes
Storage	61,247,002 -> 95,095,458 bytes
Client errors	15 (all HeadBlob , HTTP 404)

Window (UTC)	Errors
2026-10-06 02:57	13 x HeadBlob 404
2026-10-06 03:57	2 x HeadBlob 404

Successful operations near those windows included HeadBlob, PutBlob, PutManifest, GetToken, PostToken and PostExchange.

Interpretation

- The operation mix is strongly consistent with **container publishing/build activity**. PutBlob and PutManifest indicate image publishing.
- HeadBlob 404 is consistent with a client checking whether a layer blob already exists before uploading it. A 404 there means "not present, upload it" and is normal client behaviour.
- GitHub Actions is a **plausible** source because CI was actively publishing.

IMPORTANT

The telemetry does **not** prove the caller was GitHub Actions: caller identity and IP were not available. No attribution is confirmed.

SECURITY NOTE

The 404s are not treated as a security incident. Nothing in the available telemetry indicated a confirmed registry compromise. Absence of evidence in metric-only data is also not proof of absence; request-level logs would be needed for that.

Improvement (PLANNED): enable ACR diagnostic logs (repository events, login events) to a Log Analytics workspace to obtain caller identity, IP, repository and tag.

EVIDENCE

EVID-15.

22. Initial vs Final State

Area	Initial State	Final State	Security Improvement
Container base image	Debian-based NGINX	Smaller Alpine-based NGINX	Fewer packages, smaller attack surface
Vulnerability posture	Many OS-level findings	About 80% fewer reported findings (observed; reproduce before citing)	Remediation validated by re-scan
Image tagging	latest	Git commit SHA tag, digest recorded	Immutability, traceability, rollback
Registry	Docker Hub	Azure Container Registry	Azure RBAC-controlled registry in same tenant as AKS
CI/CD	Manual local build/push	GitHub Actions, 3 jobs	Repeatable, scanned pipeline
Authentication	Manual credentials	GitHub OIDC to managed identity	No stored Azure secret; short-lived tokens
Kubernetes runtime	Local Minikube	AKS	Managed control plane, Azure integration
RBAC	Basic least-privilege intent	nginx-sa /Role/RoleBinding, can-i validated	Tested least privilege
Network policy	Described as "zero trust" (inaccurate)	nginx-allow-http, TCP 8080, open ingress, open egress, accurately described	Enforced port allow; limits documented honestly
Network dataplane	Local default	Azure overlay + Cilium	Policy enforcement engine on AKS
Admission control	None	OPA Gatekeeper require-non-root	Non-compliant Deployments rejected (tested)
Container privileges	Intended	Non-root, no priv-esc enforced via securityContext	Reduced runtime privilege
Filesystem	Writable	Read-only root, emptyDir for /tmp, /var/cache/nginx	Reduces tampering surface
Observability	Snyk scan (historical)	ACR metrics investigation	Evidence-based interpretation; logging gap identified
Deployment automation	Manual	Image publish automated; AKS deploy manual	Partial; CI to AKS is future work

23. Challenges and Troubleshooting

#	Challenge	Cause	Resolution
1	Slow Trivy scan	Large vulnerability DB and scanning overhead	Optimized scanning; integrated checks into CI

#	Challenge	Cause	Resolution
2	Stale HTML	Docker/image caching and mutable tags	Explicit image versioning, later SHA-based references
3	NodePort not reachable locally	Local Kubernetes networking/isolation	<code>kubectl port-forward</code>
4	Port mismatch	<code>containerPort</code> , <code>targetPort</code> , probes disagreed	Standardized on 8080
5	NGINX default page	Wrong or stale image/content	Rebuilt and deployed the correct image
6	AKS Cilium migration	Network configuration change	Ran <code>az aks update</code> , reapplied Service
7	Gatekeeper not evaluating	Incorrect Rego target	Set <code>admission.k8s.gatekeeper.sh</code>
8	Deprecated Endpoints API	API deprecation	Used EndpointSlice
9	Azure credential/tooling issues	<code>kubectl/kubelogin</code> missing or PATH	See below

Challenge 9 detail

```
az aks install-cli
kubelogin convert-kubeconfig -l azurecli
```

`az aks install-cli` installs `kubectl` and `kubelogin`. `kubelogin convert-kubeconfig -l azurecli` configures the kubeconfig to use Azure CLI credentials for Entra-integrated AKS. When commands were not found, PATH had to be restored so the install location was included.

Quick diagnostic checklist (months later):

- `az account show` (right subscription?)
- `kubectl config current-context` (right cluster?)
- Service missing? Re-check after networking changes; reapply `K8s/service.yaml`.
- Pods not Ready? Compare `containerPort`, `targetPort`, probe port.
- Gatekeeper rejection of your own manifest? Check `runAsNonRoot` and the denial message.
- Pipeline push fails? Check federated credential subject (branch), `id-token: write`, and `AcrPush` assignment.

24. Security Validation Matrix

Control	Implementation	Test	Expected	Actual	Evidence
Trivy	FS and image scan jobs	Pipeline run; local <code>trivy image</code>	Scans execute	Executed in pipeline; initial vs follow-up scans compared (~80% observed reduction)	EVID-01, 02, 05
Gitleaks	Secret scan job	Pipeline run	Scan runs	Executed in pipeline. Findings: not recorded	EVID-05
Docker build	Build job	Build in CI and locally	Image builds	Built	EVID-03, 05

Control	Implementation	Test	Expected	Actual	Evidence
OIDC	Federated credential on managed identity	Pipeline Azure login	Login without client secret	Successful pipeline	EVID-05, 06
ACR push	AcrPush role	Pipeline push	Image in ACR	4 successful pushes in 24h metrics	EVID-04, 15
ACR immutable image	SHA tag, digest	Inspect tag/digest	Tag matches commit; digest recorded	f12214...801f / sha256:f95311...3880	EVID-04
RBAC	nginx-sa, Role, RoleBinding	kubectll auth can-i x4	yes, yes, no, no	yes, yes, no, no	EVID-08
SecurityContext	non-root, no priv-esc, read-only rootfs	Deployment inspection; pods running	Pods start and run	2/2 Running	EVID-13
NetworkPolicy	nginx-allow-http	get / describe networkpolicy	Policy present, port 8080	Present; ingress open, egress unrestricted	EVID-09
Cilium	AKS policy and dataplane	az aks update ; network profile	cilium/cilium	Configured	EVID-07, 10
Gatekeeper	Template and Constraint	Install check	Resources present	Present; 5 audit findings on system workloads	EVID-11
Gatekeeper negative test	require-non-root	kubectll create deployment gatekeeper-test -- image=nginx	Denied	Denied; NotFound afterward	EVID-12
Pods	nginx-deploy, 2 replicas	kubectll get pods -A	2/2 Ready	2/2 Ready, 0 restarts	EVID-13
Health probes	Liveness and readiness on / : 8080	Pods Ready	Ready	Ready	EVID-13
Service	nginx-service NodePort	kubectll get service	80 -> 8080	Present after reapply	EVID-14
EndpointSlice	Service-backed	get endpointslice -l ...	Ready endpoints	Validated	EVID-14
ACR observability	Metrics via Observability Agent	Metric review	Interpret activity	91 tx, 4 pushes, 15 HeadBlob 404; no attribution	EVID-15

25. Reproduction Runbook

IMPORTANT

Commands marked **Standard, verify** are conventional commands not recorded verbatim in the project. Use placeholders in angle brackets. "Actual" results are given only where recorded; otherwise "Not recorded".

Prerequisites: Windows/PowerShell, Azure CLI, kubectl, kubelogin, Docker, Git, Trivy, a GitHub repository, an Azure subscription.

Step 1. Clone repository - Purpose: obtain source. - Command: `git clone https://github.com/Lakewest1/Secure-Kubernetes-Deployment.git` - Expected: local copy. Actual: Not recorded. - Troubleshooting: check network and Git credentials.

Step 2. Build container - Purpose: build image. Command: `docker build -t <local-name>:<tag> .` (original lab: `docker build -t lakewest/my-nginx-app1:latest .`) - Expected: image built. Actual: built (EVID-03). - Troubleshooting: Docker daemon running; Dockerfile path.

Step 3. Initial Trivy scan - Purpose: baseline. Command: `trivy image lakewest/my-nginx-app1:latest` - Expected: OS package findings on the Debian image. Actual: many findings; ~1 hour in local lab (EVID-01). - Troubleshooting: slow first run due to DB download.

Step 4. Remediate base image - Purpose: reduce attack surface. Command: edit Dockerfile to a smaller Alpine-based NGINX base; remove unneeded packages; rebuild. - Expected: smaller image. Actual: done. - Troubleshooting: confirm the NGINX listens on 8080 and runs as non-root (**verify against Dockerfile**).

Step 5. Follow-up scan - Purpose: validate remediation. Command: `trivy image <remediated-image>` - Expected: fewer findings. Actual: ~80% reduction observed (EVID-02). - Troubleshooting: record scanner and DB versions to make results comparable.

Step 6. Create Azure resources - Purpose: resource group. Command: `az group create --name SOC-lab-01 --location eastus` **Standard, verify** - Expected: group created. Actual: environment exists. - Troubleshooting: `az account set --subscription "SOC-lab"`.

Step 7. Configure ACR - Purpose: registry. Command: `az acr create --resource-group SOC-lab-01 --name securek8sacr01 --sku <sku>` **Standard, verify** - Expected: login server `securek8sacr01.azurecr.io`. Actual: exists. - Troubleshooting: ACR names are globally unique.

Step 8. Configure AKS - Purpose: cluster with Azure RBAC, managed identity, OIDC issuer, overlay, Cilium. Command: `az aks create --resource-group SOC-lab-01 --name secure-k8s-aks01 --node-count 1 --node-vm-size Standard_D2_v3 --enable-aad --enable-azure-rbac --enable-oidc-issuer --network-plugin azure --network-plugin-mode overlay --network-dataplane cilium --network-policy cilium` **[Standard, verify flags for your CLI version]**. The project reached Cilium via `az aks update` (Step 8a below). - Step 8a, recorded command:

```
az aks update `
  --resource-group "SOC-lab-01" `
  --name "secure-k8s-aks01" `
  --network-policy cilium `
  --network-dataplane cilium
```

- Expected: cluster running. Actual: Kubernetes 1.35.8 validated; Cilium migration done (EVID-07, 10).
- Troubleshooting: Service may vanish after migration; reapply (Step 23).

Step 9. AKS to ACR AcrPull - Purpose: let kubelet pull images. Command: `az aks update -n secure-k8s-aks01 -g SOC-lab-01 --attach-acr securek8sacr01` Standard, verify, or assign `AcrPull` to the kubelet identity. - Expected: `AcrPull` on kubelet identity. Actual: configured. - Troubleshooting: `ImagePullBackOff` indicates missing `AcrPull`.

Step 10. Configure GitHub OIDC - Purpose: passwordless CI. Create managed identity `github-actions-secure-k8s`; federated credential `github-actions-main` with issuer `https://token.actions.githubusercontent.com`, subject `repo:Lakewest1/Secure-Kubernetes-Deployment:ref:refs/heads/main`, audience `api://AzureADTokenExchange`; assign `AcrPush` on the registry. Commands: Standard, verify `az identity create`, `az identity federated-credential create`, `az role assignment create`. - Expected: Azure accepts GitHub tokens. Actual: pipeline logs in (EVID-06). - Troubleshooting: subject must match exactly (case, branch).

Step 11. Configure GitHub Actions secrets - Purpose: supply identifiers. Add repository secrets/variables for client ID, tenant ID, subscription ID (identifiers, not passwords). Names depend on your workflow. - Expected: `azure/login` succeeds. Actual: succeeded. - Troubleshooting: wrong IDs cause AADSTS errors.

Step 12. Run CI pipeline - Purpose: scan, build, publish. Command: push to `main` (or run the workflow). - Expected: three jobs pass. Actual: success (EVID-05). - Troubleshooting: missing `id-token: write`; non-`main` branch will not match the subject.

Step 13. Validate ACR image - Purpose: confirm publication. Command: `az acr repository show-tags --name securek8sacr01 --repository secure-nginx` Standard, verify - Expected: SHA tag `f12214058cd0e11d8b77dbe5ce0de799a777801f`; digest `sha256:f95311cd1cc733c91c5b67b26c15f5dde5369cc1d6f8075871cfd9a96df93880`. Actual: as recorded (EVID-04). - Troubleshooting: `az acr login` if needed.

Step 14. Get AKS credentials - Command: `az aks get-credentials --resource-group SOC-lab-01 --name secure-k8s-aks01` - Expected: kubeconfig entry. Actual: Not recorded. - Troubleshooting: `az login`, correct subscription.

Step 15. Configure kubelogin - Commands: `az aks install-cli`; `kubelogin convert-kubeconfig -l azurecli` - Expected: `kubectl get nodes` works. Actual: worked after PATH restore. - Troubleshooting: restore PATH; reopen shell.

Step 16. Install Gatekeeper - Command: Standard, verify apply the official Gatekeeper release manifest or Helm chart, pinned to a specific version; then `kubectl get pods -n gatekeeper-system`. - Expected: controller pods Running. Actual: installed (EVID-11). - Troubleshooting: webhook not ready delays constraint creation.

Step 17. Apply ConstraintTemplate - Command: `kubectl apply -f policies/gatekeeper/constraint-template.yaml` - Expected: template `k8srequirednonroot` created. Actual: applied after target fix to `admission.k8s.gatekeeper.sh`. - Troubleshooting: wrong target means policy does not evaluate.

Step 18. Apply Constraint - Command: `kubectl apply -f policies/gatekeeper/constraint.yaml` - Expected: `require-non-root` created. Actual: applied. - Troubleshooting: constraint kind must match the template; apply template first.

Step 19. Apply RBAC - Command: `kubectl apply -f K8s/Rbac.yaml` - Expected: ServiceAccount, Role, RoleBinding. Actual: applied. - Troubleshooting: filename case on case-sensitive systems.

Step 20. Apply NetworkPolicy - Command: `kubectl apply -f K8s/Network-policy.yaml` - Expected: `nginx-allow-http` created. Actual: applied (EVID-09). - Troubleshooting: confirm selector `app=nginx-deploy` matches pod labels.

Step 21. Apply Secret - Command: `kubectl apply -f K8s/secret.yaml` - Expected: `demo-secret`. Actual: applied; demonstration data only. - Troubleshooting: never put real secrets in Git.

Step 22. Apply Deployment - Command: `kubectl apply -f K8s/deployment.yaml` - Expected: 2 replicas, Gatekeeper admits. Actual: 2/2 Ready. - Troubleshooting: rejected by Gatekeeper means missing `runAsNonRoot` ; `ImagePullBackOff` means `AcrPull` or image reference.

Step 23. Apply Service - Command: `kubectl apply -f K8s/service.yaml` - Expected: `nginx-service`, 80 -> 8080, NodePort 30007. Actual: reapplied after Cilium migration. - Troubleshooting: re-check after any networking change.

Step 24. Validate Pods - Command: `kubectl get pods -A`; `kubectl get deployment` - Expected: 2/2 Ready. Actual: 2/2 Ready, 0 restarts (EVID-13). - Troubleshooting: `kubectl describe pod`, `kubectl logs`.

Step 25. Validate RBAC - Commands: the four `kubectl auth can-i ... --as=system:serviceaccount:default:nginx-sa` commands (Section 13). - Expected/Actual: yes, yes, no, no (EVID-08). - Troubleshooting: verify Role and binding namespace.

Step 26. Validate NetworkPolicy - Commands: `kubectl get networkpolicy`; `kubectl describe networkpolicy nginx-allow-http` - Expected: policy listed. Actual: present (EVID-09). - Troubleshooting: policy ignored means no enforcing engine.

Step 27. Validate Gatekeeper - Commands: `kubectl get constrainttemplate`; `kubectl get constraints` `Standard, verify` - Expected: template and constraint present; audit lists violations. Actual: 5 audit findings on system workloads. - Troubleshooting: audit results appear after the audit interval.

Step 28. Negative admission test - Command: `kubectl create deployment gatekeeper-test --image=nginx`; then `kubectl get deployment gatekeeper-test` - Expected: denied; NotFound. Actual: denied; NotFound (EVID-12). - Troubleshooting: if created, webhook or constraint is not active; delete the test deployment.

Step 29. Validate final runtime - Commands: `kubectl get service`; `kubectl get endpointslice -l kubernetes.io/service-name=nginx-service`; optionally `kubectl port-forward service/nginx-service 8080:80` and browse `http://localhost:8080` . - Expected: Service with ready endpoints, page served. Actual: validated (EVID-14). - Troubleshooting: empty `EndpointSlice` means selector or readiness failure.

Step 30. Review ACR telemetry - Purpose: interpret registry activity (Section 21). Use Azure portal metrics. - Expected/Actual: 91 transactions, 4 pushes, 15 HeadBlob 404 (EVID-15). - Troubleshooting: enable diagnostic logs for caller attribution.

Step 31. Capture evidence - Capture each EVID screenshot into the folder structure (Section 20). Crop to the relevant output; ensure command and result are both visible.

Step 32. Stop AKS - Command: `az aks stop --resource-group SOC-lab-01 --name secure-k8s-aks01` `Standard, verify` - Expected: control plane and nodes stopped. Troubleshooting: costs for other resources (ACR) continue.

Step 33. Optionally delete ACR - Command: `az acr delete --name securek8sacr01 --resource-group SOC-lab-01` `Standard, verify` - Warning: destroys stored images. Skip unless decommissioning.

26. Command Reference

Only commands used in the project or listed as `Standard, verify` appear here.

Azure authentication

```
az login
az account show
az account set --subscription "SOC-lab" # Standard, verify
az aks install-cli
kubelogin convert-kubeconfig -l azurecli
```

Azure resource discovery (all Standard, verify)

```
az group show --name SOC-lab-01
az aks show --resource-group SOC-lab-01 --name secure-k8s-aks01
az acr show --name securek8sacr01
```

AKS

```
az aks update `
  --resource-group "SOC-lab-01" `
  --name "secure-k8s-aks01" `
  --network-policy cilium `
  --network-dataplane cilium
az aks get-credentials --resource-group SOC-lab-01 --name secure-k8s-aks01
az aks stop --resource-group SOC-lab-01 --name secure-k8s-aks01 # Standard, verify
```

ACR (Standard, verify)

```
az acr login --name securek8sacr01
az acr repository show-tags --name securek8sacr01 --repository secure-nginx
```

kubectl

```
kubectl get pods -A
kubectl get nodes
kubectl get deployment
kubectl get service
kubectl get endpointslice
kubectl get endpointslice -l kubernetes.io/service-name=nginx-service
kubectl apply -f K8s/service.yaml
kubectl port-forward service/nginx-service 8080:80
```

RBAC

```
kubectl auth can-i get pods --as=system:serviceaccount:default:nginx-sa
kubectl auth can-i list pods --as=system:serviceaccount:default:nginx-sa
kubectl auth can-i delete pods --as=system:serviceaccount:default:nginx-sa
kubectl auth can-i create deployments --as=system:serviceaccount:default:nginx-sa
```

NetworkPolicy

```
kubectl get networkpolicy
kubectl describe networkpolicy nginx-allow-http
```

Gatekeeper

```
kubectl create deployment gatekeeper-test --image=nginx
kubectl get deployment gatekeeper-test
```

Docker

```
docker build -t lakewest/my-nginx-app1:latest .
docker push lakewest/my-nginx-app1:latest # HISTORICAL (Docker Hub)
```

Trivy

```
trivy image lakewest/my-nginx-app1:latest
```

Historical optimization: `--scanners vuln` limits scanning to vulnerabilities.

Git

```
git clone https://github.com/Lakewest1/Secure-Kubernetes-Deployment.git
```

GitHub Actions validation: inspect the Actions run for the three jobs; confirm the `permissions` block; confirm the federated credential subject matches `refs/heads/main`.

27. Configuration Reference

Contents of these files were not supplied for this rewrite. Entries below describe documented purpose and constraints. Confirm against the repository.

File	Purpose	Key security controls	Depends on	Safe to change	Must stay consistent
K8s/ deployment.yaml	nginx-deploy, 2 replicas	securityContext, emptyDir mounts, limits, probes, SHA image	ACR image, ServiceAccount, Gatekeeper	Image tag/digest, replicas	Port 8080 across containerPort/probes/Service targetPort; label <code>app=nginx-deploy</code> ; <code>runAsNonRoot: true</code>
K8s/Rbac.yaml	nginx-sa, nginx-role, nginx-binding	Pods get/list only	Namespace default	Resource scope (tighten)	Binding subject/role names
K8s/Network-policy.yaml	nginx-allow-http	TCP 8080 ingress rule	Cilium engine	Add source restrictions, egress rules	Pod selector matches labels; port 8080

File	Purpose	Key security controls	Depends on	Safe to change	Must stay consistent
K8s/service.yaml	nginx-service NodePort 80 -> 8080	Exposes only service port	Pod labels	Service type, NodePort	targetPort 8080; selector
K8s/secret.yaml	demo-secret	None (base64 only)	n/a	Replace with managed secrets	Name if referenced
policies/gatekeeper/constraint-template.yaml	k8srequirednonroot	Rego, target admission.k8s.gatekeeper.sh	Gatekeeper	Rego logic	Target; kind matches constraint
policies/gatekeeper/constraint.yaml	require-non-root	Enforcement scope	Template	match scope (exclude system namespaces)	Kind matches template
.github/workflows/<workflow-file>	Three-job pipeline	Scans, OIDC, permissions	Azure identity, ACR	Triggers, scan thresholds	Branch = federated subject; id-token: write
Dockerfile	Image build	Minimal Alpine-based NGINX	Content, nginx config	Content, package set	Listens on 8080; non-root user
README.md	Project entry point	Accurate claims	All of the above	Wording	Must match this report

28. Production Readiness Gap Analysis

This project is a **production-aligned security engineering demonstration**. It is not production-ready.

Current Control	Current Implementation	Production Gap	Recommended Improvement	Priority
NetworkPolicy	Open ingress to 8080, open egress	No source restriction, no default deny, no egress control	Default-deny per namespace; allow known sources; restrict egress	High
Secrets	Demo base64 Secret	Not secret management	Key Vault + CSI/External Secrets; encryption at rest review	High
CI to AKS deployment	Manual kubectl apply	Manual drift, no audit trail	Pipeline deploy with OIDC and scoped AKS role, or GitOps	High
Image signing	None	No provenance verification	Cosign/Sigstore; admission verification	Medium
SBOM	None	No dependency inventory	Generate and store SBOM in CI	Medium
Runtime monitoring	None	No runtime threat detection	Azure Monitor / Defender, runtime tooling	High
Gatekeeper scope	One policy, broad match	Noise from system namespaces; single rule	Scope/exclude; add policies (registry allow-list, no latest, limits)	Medium

Current Control	Current Implementation	Production Gap	Recommended Improvement	Priority
Pod Security Standards	Not configured	No namespace-level baseline	Enforce <code>restricted</code> where feasible	Medium
IaC	Manual/CLI	Not reproducible by code	Terraform/Bicep	Medium
Logging	ACR metrics only	No caller attribution	ACR diagnostic logs, AKS audit/control-plane logs to Log Analytics	High
Alerting	None	No detection response	Alerts on failed auth, anomalous pushes	Medium
Rollback	Manual redeploy of prior SHA	No automated rollback	Deployment strategy, versioned manifests	Medium
Environment separation	Single environment	No dev/stage/prod isolation	Separate subscriptions/clusters/registries	High
Approval gates	None	No review before deploy	Protected environments, required reviewers	Medium
Cluster resilience	1 node	No node-level HA	Multi-node, zones	Medium
Branch protection	Not documented	OIDC trust depends on <code>main</code> integrity	Enforce reviews and protection on <code>main</code>	High

29. Future Roadmap

Phase	Scope	Status
1	Current validated implementation (Sections 3 to 21)	IMPLEMENTED
2	CI to AKS automated deployment with scoped identity and approval	PLANNED
3	Secrets management: Azure Key Vault, CSI Secret Store or External Secrets	PLANNED
4	Image signing (Cosign/Sigstore) and SBOM	PLANNED
5	Runtime monitoring: Azure Monitor; possibly Prometheus and Grafana	PLANNED
6	Advanced Gatekeeper policies; Pod Security Standards	PLANNED
7	Infrastructure as Code (Terraform)	PLANNED
8	Production environment separation	PLANNED

None of the above is implemented.

30. Interview Defense Guide

- 1. Why Alpine?** To reduce attack surface: fewer packages and libraries (the Debian image carried `libtiff`, `libheif`, `libxslt` transitively). The benefit was validated by re-scanning, not assumed.
- 2. Why is a smaller image not automatically secure?** Fewer packages mean fewer potential vulnerabilities, but it says nothing about configuration, app code, or the remaining packages. Alpine has its own CVEs and musl compatibility trade-offs. Security comes from removal plus verification plus runtime hardening.
- 3. Why SHA tags?** `latest` is mutable and caused stale content. A commit SHA ties each image to source and enables deterministic rollback.
- 4. Why digests?** A digest is a content hash; a tag can be repointed. The digest is the immutable identity (`sha256:f95311...`).
- 5. Why GitHub OIDC?** No stored Azure secret; short-lived tokens; trust restricted to this repo and the `main` branch via the federated credential subject.
- 6. Why separate AcrPush from AcrPull?** CI needs write; the cluster needs read. A compromised node should not be able to overwrite images, so the kubelet identity gets only `AcrPull`.
- 7. Why a ServiceAccount?** To give the workload its own identity rather than the default one, so permissions can be scoped and tested. In this project it demonstrates the pattern.
- 8. Why is get/list pods least privilege?** Read-only, one resource type, one namespace via a Role. `can-i` confirmed delete and create deployments are denied.
- 9. Does NGINX need Kubernetes API permissions?** No. The static site does not call the API. The Role demonstrates restricted authorization, and I do not claim otherwise. Production would use a ServiceAccount with no bindings (and `automountServiceAccountToken: false` where applicable; not claimed as implemented).
- 10. What does `runAsNonRoot` do?** The kubelet refuses to start the container as UID 0, limiting damage if NGINX is compromised.
- 11. Why disable privilege escalation?** It sets `no_new_privs`, preventing setuid-style escalation inside the container.
- 12. Why `readOnlyRootFilesystem`?** It prevents tampering with the container filesystem and limits persistence of dropped files.
- 13. Why are `emptyDir` volumes required?** NGINX needs writable temp/cache paths (`/tmp`, `/var/cache/nginx`). Giving it only those keeps the rest read-only.

14. What does the NetworkPolicy enforce? Ingress to TCP 8080 on pods labelled `app=nginx-deploy`, from any source. Egress is unrestricted.

15. Why not call it zero trust? Zero trust implies identity-based segmentation and default deny. This policy allows any source and does not control egress. Calling it zero trust would be an overclaim.

16. Why Cilium? It provides the policy enforcement engine and eBPF dataplane on AKS, enabled through supported AKS settings (`--network-policy cilium --network-dataplane cilium`) on Azure overlay networking.

17. What does Gatekeeper do? It is an admission webhook evaluating Rego constraints before objects are stored, and it audits existing objects.

18. How did you prove enforcement? The negative test: `kubectl create deployment gatekeeper-test --image=nginx` was denied with `[require-non-root] ...`, and `kubectl get deployment gatekeeper-test` returned `NotFound`.

19. Why five violations? The constraint matched Deployments broadly, and audit flagged AKS-managed system workloads (konnnectivity-agent, konnectivity-agent-autoscaler, coredns, coredns-autoscaler, cilium-operator). It is audit output, not an application flaw.

20. Why not modify system workloads? They are platform-managed; edits can be reverted or destabilize the cluster. The right fix is scoping the constraint, not changing managed components.

21. Why EndpointSlice? `Endpoints` is deprecated; `EndpointSlice` is the current, scalable API.

22. What caused the HeadBlob 404s? Most consistent with a client checking whether layer blobs exist before upload (a 404 means "upload needed"), alongside successful `PutBlob`/`PutManifest`.

23. Can you prove they came from GitHub Actions? No. Metric-level telemetry had no caller identity or IP. It is plausible given active CI publishing, not confirmed. I would enable ACR diagnostic logs to attribute.

24. Why isn't automated AKS deployment implemented? It was scoped out. Safe CI deployment needs a scoped AKS role for the pipeline, environment protection and approvals. It is Phase 2.

25. What would you change before production? Restrict `NetworkPolicy` and add default deny; replace demo `Secret` with Key Vault integration; automate deployment with approvals; add signing and SBOM; enable logging and alerting; scope `Gatekeeper` and add policies; adopt IaC; separate environments; protect `main`.

31. Portfolio Value

Domain	What this project shows (within its scope)
Cloud security	Azure RBAC-based registry access, managed identities
DevSecOps	Scan-build-scan-publish pipeline with secret scanning
Kubernetes security	<code>securityContext</code> , RBAC, probes, limits, admission policy
Container security	Findings analysis and dependency-driven remediation
CI/CD security	Passwordless OIDC, minimal workflow permissions, SHA tagging
Cloud IAM / workload identity	Federated credential scoping; <code>AcrPush</code> vs <code>AcrPull</code> separation
Policy-as-code	Rego constraint with negative test
Network security	<code>NetworkPolicy</code> on Cilium, with limits honestly stated

Domain	What this project shows (within its scope)
Vulnerability management	Baseline, remediate, re-scan
Security automation	Automated scans and image publishing (not deployment)
Cloud observability	Metric-based investigation with disciplined attribution limits
Troubleshooting	Documented causes and fixes across networking, tooling and policy

It demonstrates engineering method and honest scoping. It does not demonstrate production operations, multi-environment management or runtime threat detection.

32. Conclusion

The project began as a basic container and Kubernetes deployment. Trivy exposed a bloated Debian image, and root-cause analysis led to a smaller Alpine-based image, validated by follow-up scanning. The workflow moved to Azure, introduced GitHub OIDC so CI could publish to ACR without a stored secret, added automated Trivy and Gitleaks checks, and began publishing SHA-tagged, digest-identified images. On AKS, the workload was hardened with a restrictive securityContext, probes, limits and RBAC; Cilium was introduced for network policy and dataplane; and OPA Gatekeeper enforced a policy that a negative test proved was active. The ACR telemetry investigation was reported conservatively, stating what the data could and could not show.

The strongest characteristic is not the number of tools but the lifecycle:

```
BUILD -> SCAN -> ANALYZE -> FIX -> AUTHENTICATE -> PUBLISH -> DEPLOY
      -> HARDEN -> ENFORCE -> VALIDATE -> OBSERVE -> DOCUMENT
```

Known limits remain: manual AKS deployment, an open NetworkPolicy, a demonstration Secret, and no signing, SBOM, runtime monitoring or IaC. These are recorded in Sections 28 and 29 rather than hidden.

33. Reality Audit

Item searched for	Result in this report
Snyk continuous monitoring	Absent; Snyk appears only as HISTORICAL (Section 6)
Zero trust / deny all	Only in statements saying the policy is neither
Minikube final deployment	HISTORICAL only
Docker Hub final registry	HISTORICAL only
EKS, Prometheus, Grafana, Kyverno, Sentinel	PLANNED or absent; not implemented
Automated AKS deployment	Stated as NOT implemented
Production-ready	Stated as not production-ready
Unverified content	Items flagged Standard, verify , "Not recorded", or "Verify"

Items needing your confirmation against the repository: exact workflow file name and Trivy failure thresholds; whether the Deployment references tag or digest; Dockerfile base image and non-root setup; Gitleaks results; pull count and caller details.

Figure Frames for Screenshots

Each frame below is a placeholder. Insert the original, unedited screenshot, then keep the caption only if the screenshot shows what the note says.

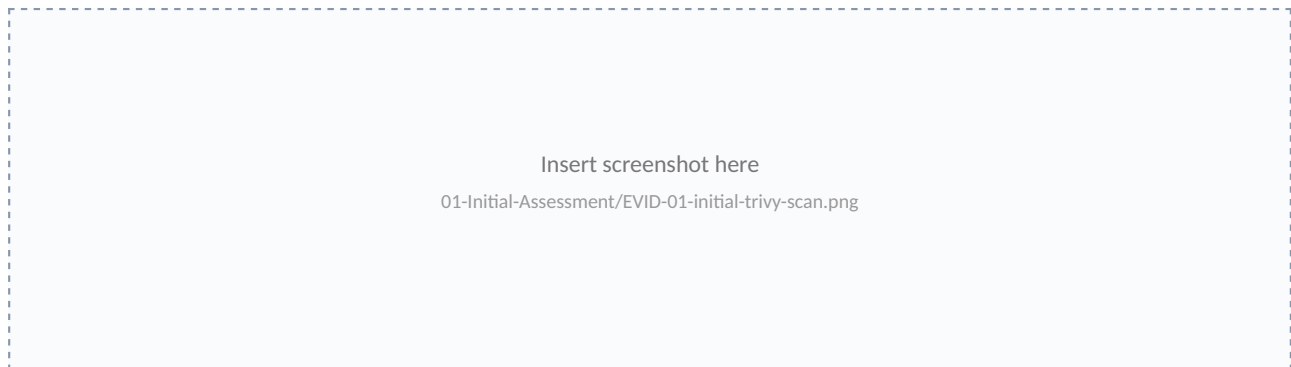


Figure 1 — Initial Trivy scan of the Debian-based image EVID-01

What to notice: Baseline findings before remediation (only if visible in the screenshot).

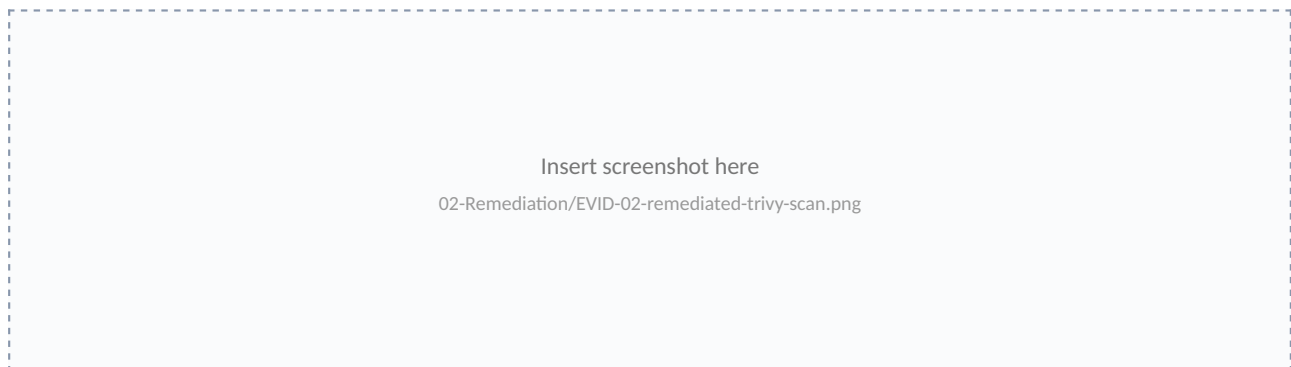


Figure 2 — Follow-up Trivy scan of the Alpine-based image EVID-02

What to notice: Reduced findings after remediation (only if visible in the screenshot).

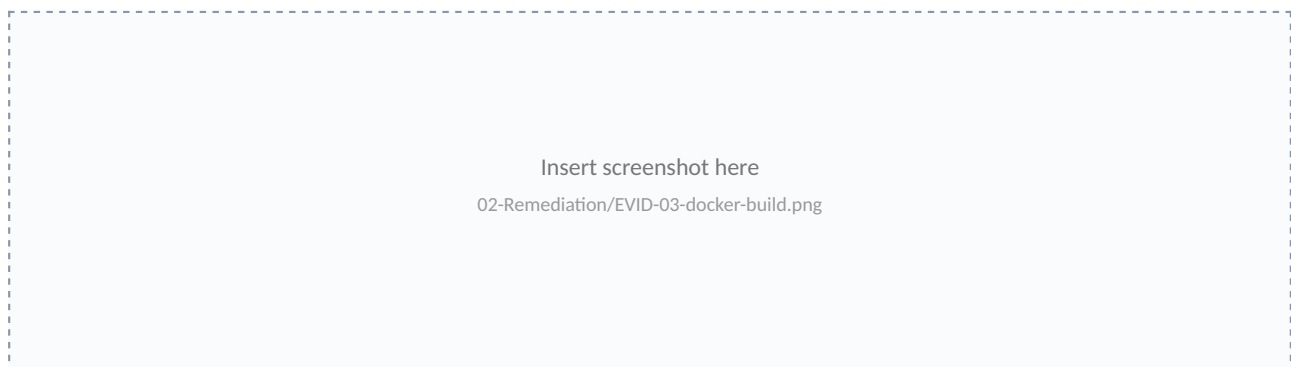


Figure 3 — Docker image build output EVID-03

What to notice: Image built successfully (only if visible in the screenshot).

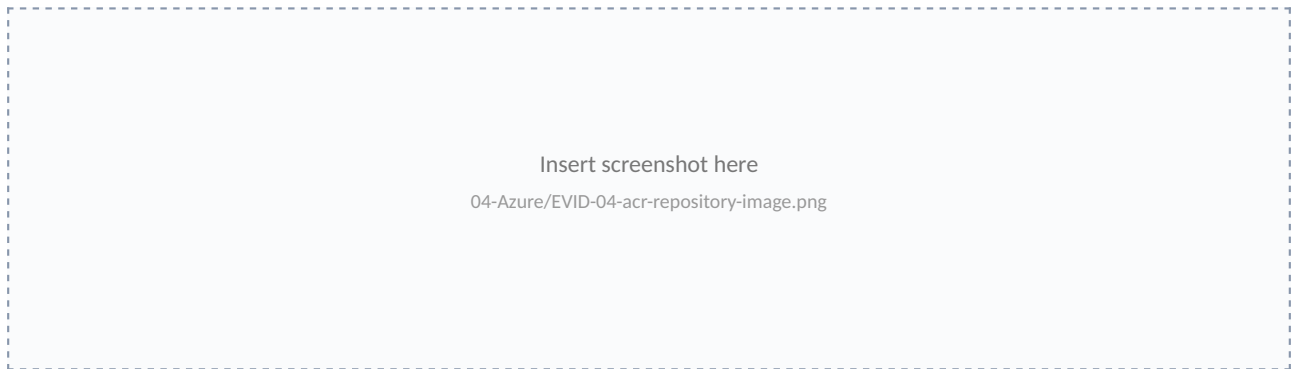


Figure 4 — Image in ACR [EVID-04](#)

What to notice: Repository, tag and (if shown) digest (only if visible in the screenshot).

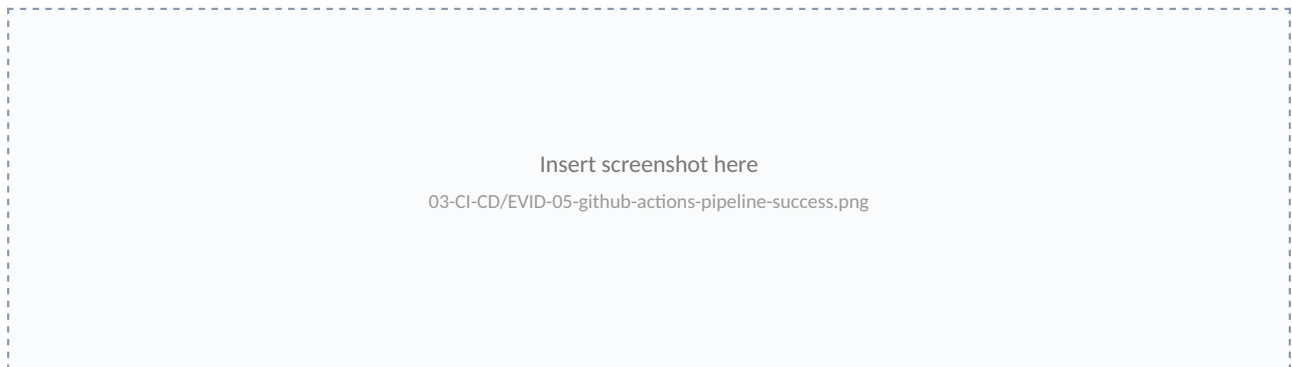


Figure 5 — Three-job pipeline completed [EVID-05](#)

What to notice: Jobs ran and passed (only if visible in the screenshot).

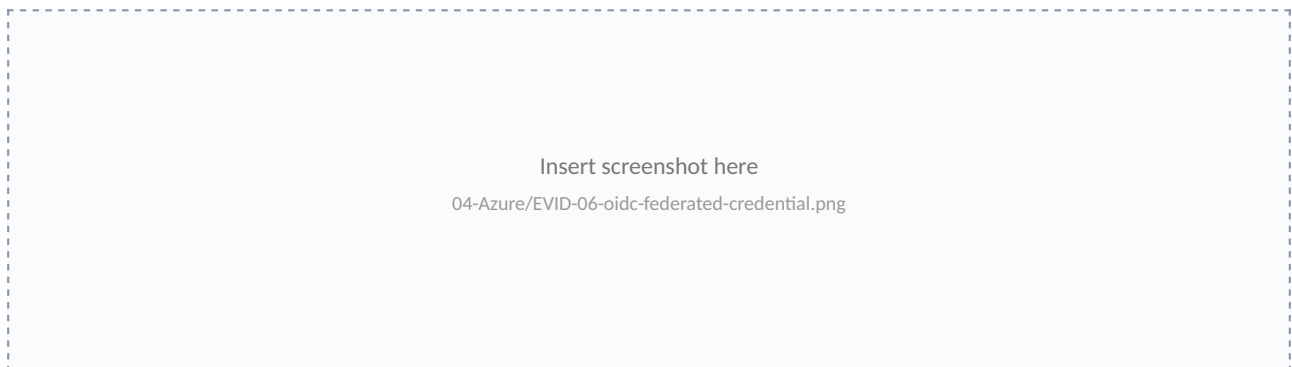


Figure 6 — Managed identity federated credential [EVID-06](#)

What to notice: Issuer, subject, audience settings (only if visible in the screenshot).

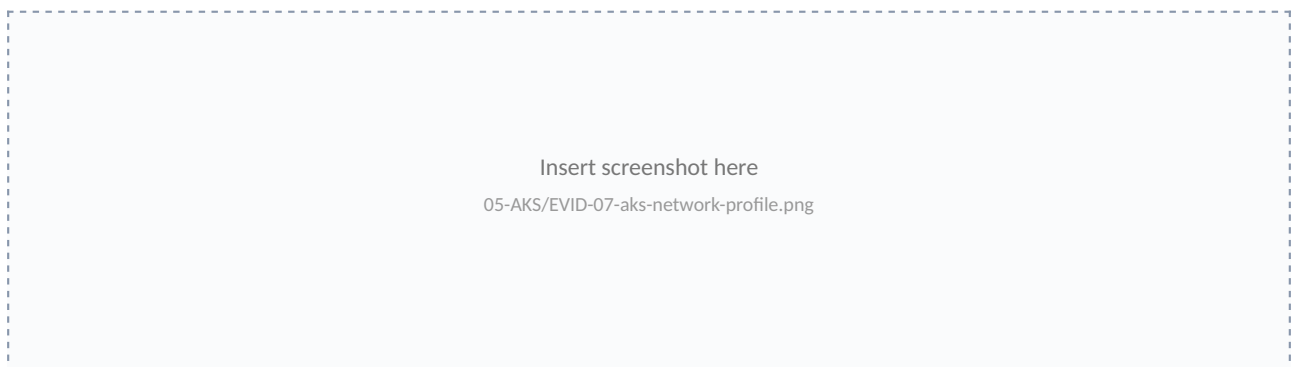


Figure 7 — AKS network profile [EVID-07](#)

What to notice: azure/overlay/cilium/cilium (only if visible in the screenshot).

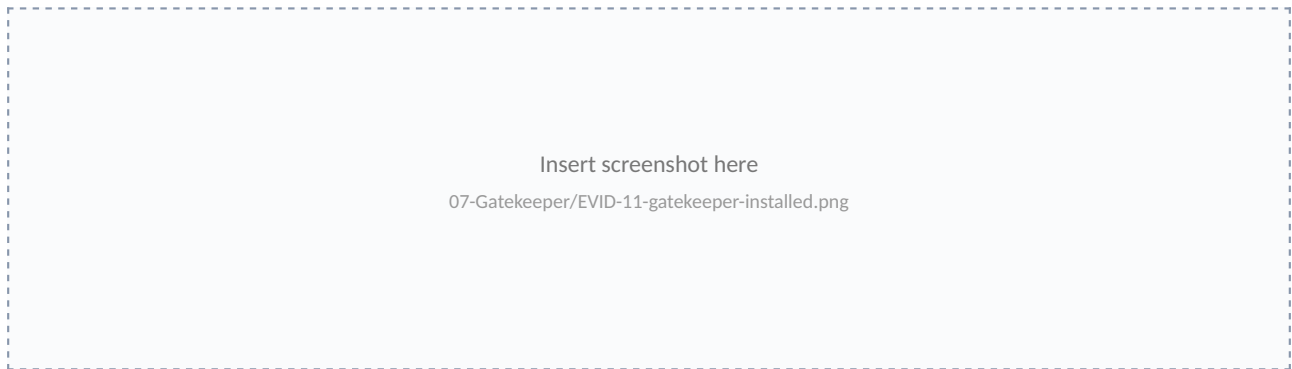


Figure 8 — Gatekeeper, template, constraint [EVID-11](#)

What to notice: Components present (only if visible in the screenshot).

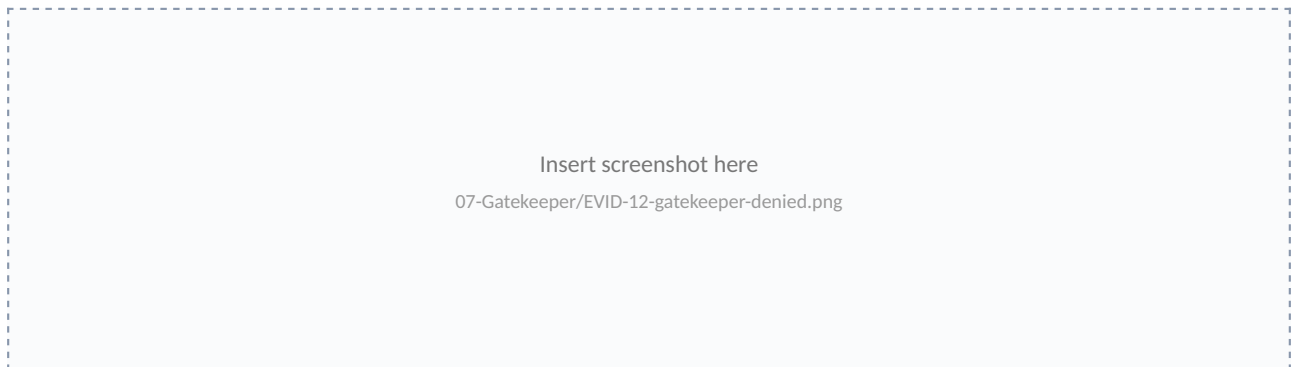


Figure 9 — Denied Deployment and NotFound [EVID-12](#)

What to notice: Enforcement working (only if visible in the screenshot).

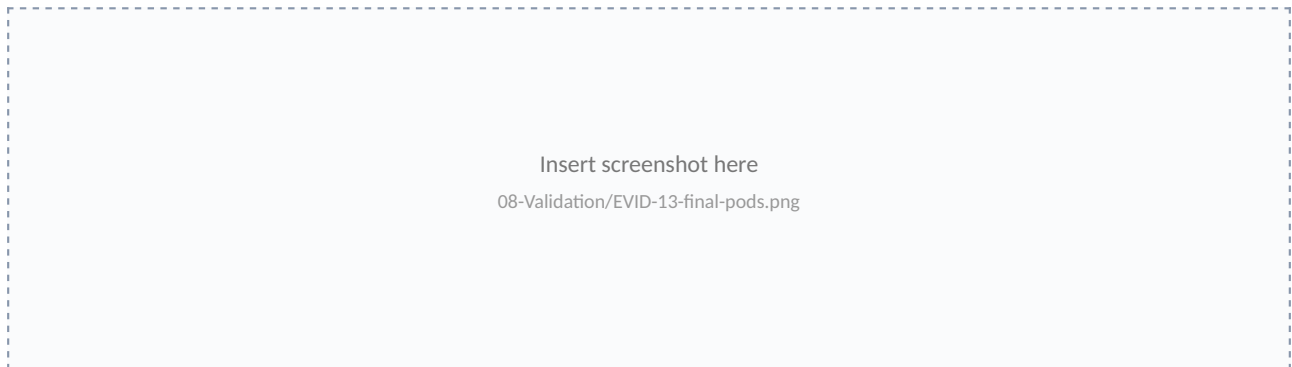


Figure 10 — Final pods and deployment [EVID-13](#)

What to notice: 2/2 Ready, Running, restarts 0 (only if visible in the screenshot).

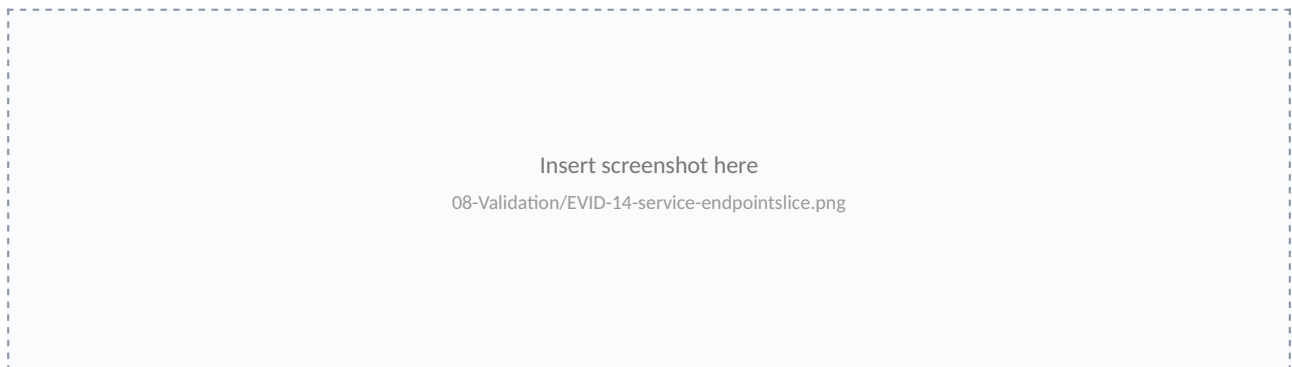


Figure 11 — Service and EndpointSlice [EVID-14](#)

What to notice: Service present, endpoints ready (only if visible in the screenshot).

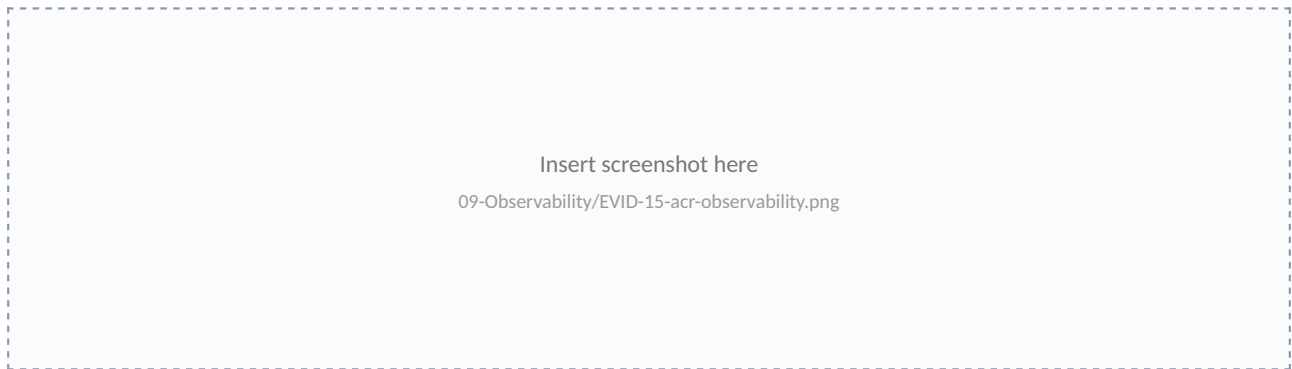


Figure 12 — ACR metrics view [EVID-15](#)

What to notice: Transactions, pushes, 404 breakdown (only if visible in the screenshot).

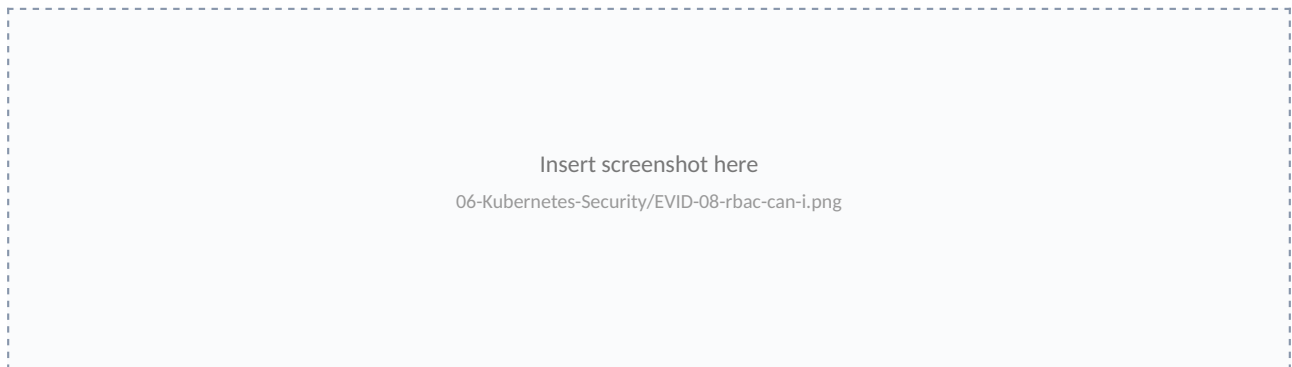


Figure 13 — Hardening [EVID-08](#)

What to notice: RBAC (only if visible in the screenshot).

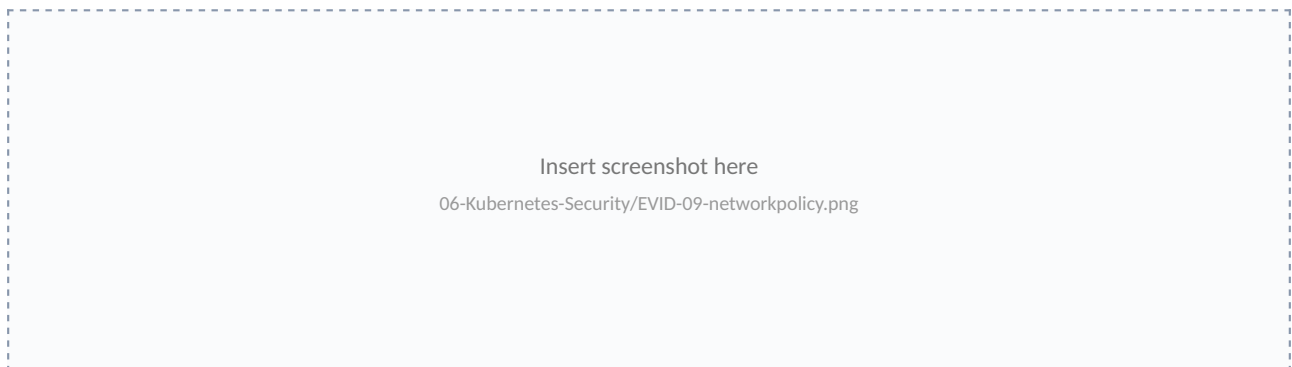


Figure 14 — Hardening [EVID-09](#)

What to notice: NetworkPolicy (only if visible in the screenshot).

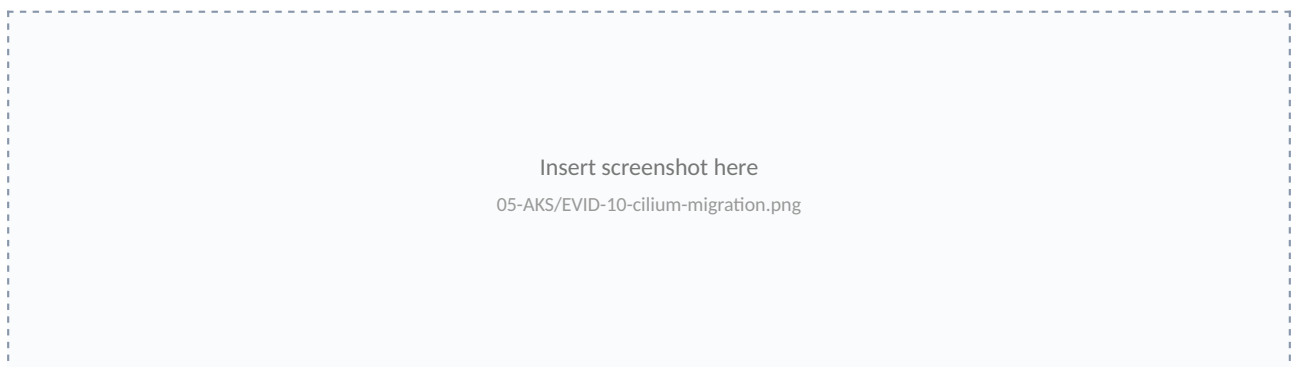


Figure 15 — AKS [EVID-10](#)

What to notice: Migration (only if visible in the screenshot).

34. Evidence Index

ID	File	Phase	Supports	Matrix row
EVID-01	01-Initial-Assessment/EVID-01-initial-trivy-scan.png	Assessment	Initial findings	Trivy
EVID-02	02-Remediation/EVID-02-remediated-trivy-scan.png	Remediation	Reduced findings	Trivy
EVID-03	02-Remediation/EVID-03-docker-build.png	Remediation	Build	Docker build
EVID-04	04-Azure/EVID-04-acr-repository-image.png	Publish	Tag/digest	ACR push, immutable image
EVID-05	03-CI-CD/EVID-05-github-actions-pipeline-success.png	CI/CD	Pipeline	Trivy, Gitleaks, OIDC
EVID-06	04-Azure/EVID-06-oidc-federated-credential.png	Identity	OIDC config	OIDC
EVID-07	05-AKS/EVID-07-aks-network-profile.png	AKS	Network profile	Cilium
EVID-08	06-Kubernetes-Security/EVID-08-rbac-can-i.png	Hardening	RBAC	RBAC
EVID-09	06-Kubernetes-Security/EVID-09-networkpolicy.png	Hardening	NetworkPolicy	NetworkPolicy
EVID-10	05-AKS/EVID-10-cilium-migration.png	AKS	Migration	Cilium
EVID-11	07-Gatekeeper/EVID-11-gatekeeper-installed.png	Enforcement	Install	Gatekeeper
EVID-12	07-Gatekeeper/EVID-12-gatekeeper-denied.png	Enforcement	Denial	Negative test
EVID-13	08-Validation/EVID-13-final-pods.png	Validation	Pods	Pods, probes, SecurityContext
EVID-14	08-Validation/EVID-14-service-endpointslice.png	Validation	Service	Service, EndpointSlice
EVID-15	09-Observability/EVID-15-acr-observability.png	Observe	ACR telemetry	ACR observability

End of report.